

VPR Tutorial Architecture Guide

A Practical Codebase Walkthrough for New Developers

Project Documentation

April 20, 2026

Purpose

This document is a practical onboarding guide to the current codebase. It is written for a junior developer who wants to understand:

- where the code starts,
- how the original benchmark pipeline and the newer live pipeline relate,
- which files are responsible for which parts of the system,
- and where to make changes when extending the project.

This guide is intentionally tutorial-style: it explains the important flow first, then tours the key files with short code snippets and commentary.

1 Big Picture

The repository now has two main paths that share the same VPR idea.

- **Benchmark path:** load a dataset, extract descriptors, compute a similarity matrix, apply matching, then compute metrics and visualizations.
- **Live path:** build a reusable reference map, then localize camera frames or video frames against that map in real time.

2 Where To Start Reading

If you are new to the project, read these files in this order:

1. `demo.py`
2. `datasets/load_dataset.py`
3. `matching/matching.py`
4. `evaluation/metrics.py`

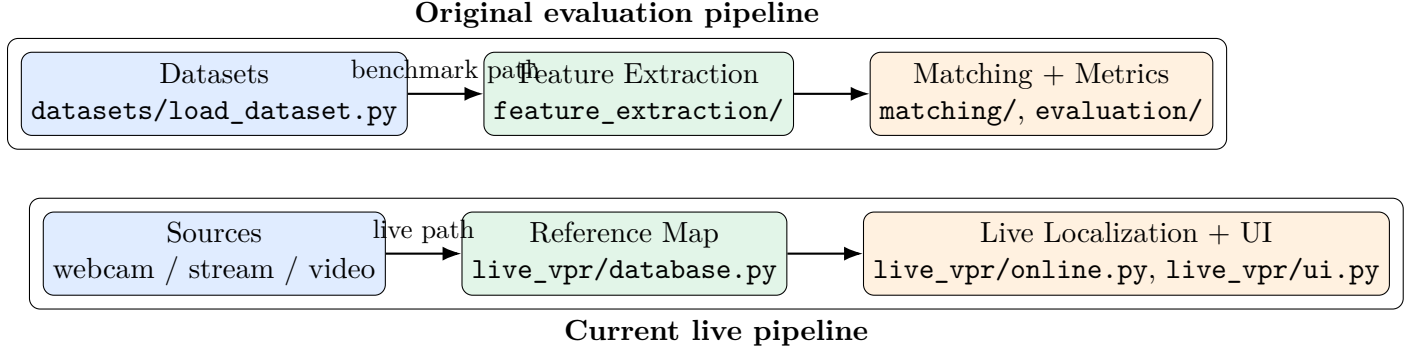


Figure 1: Repository overview: two main paths share descriptor extraction ideas, but only the benchmark path computes evaluation metrics during runtime.

5. `test_campus_dataset.py`
6. `live_vpr_test.py`
7. the modules under `live_vpr/`

This order is useful because it teaches the simple benchmark flow first and then the more modular live system.

3 Architecture Diagrams

3.1 Benchmark Flow

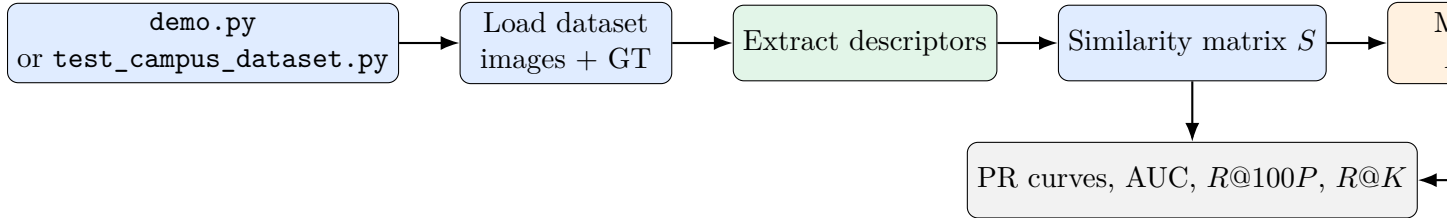


Figure 2: The benchmark path computes a full similarity matrix and then evaluates it against ground truth.

3.2 Live Flow

3.3 Module-Level Ownership

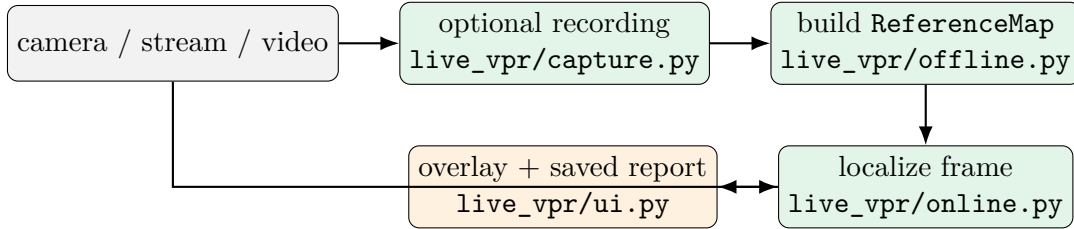


Figure 3: The live path builds a reusable map once and then performs repeated single-frame localization against it.

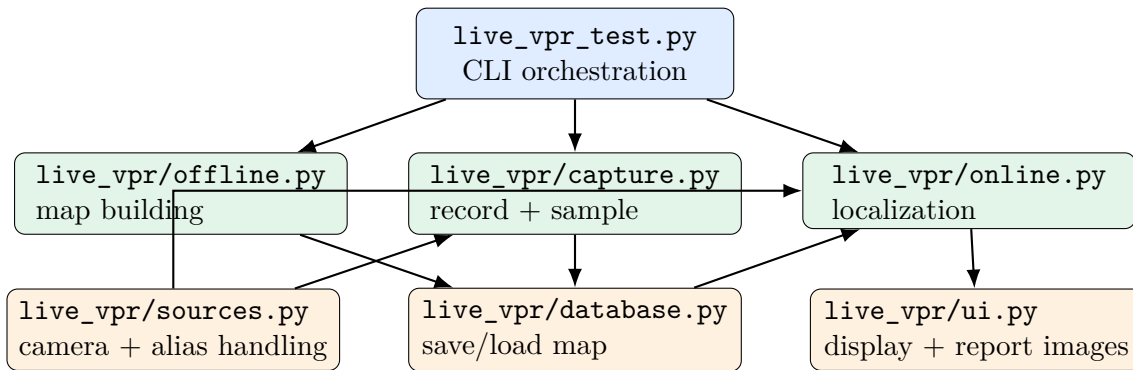


Figure 4: Module-level view of the live system. `live_vpr_test.py` coordinates the workflow; the logic lives in smaller modules.

4 Code Tour

4.1 demo.py

What it does. This is the original tutorial entrypoint. It demonstrates the full benchmark evaluation loop on the built-in datasets.

Why it matters. If you understand this file, you understand the basic VPR idea used everywhere else in the repo.

Listing 1: Core benchmark pattern in demo.py

```
imgs_db, imgs_q, GThard, GTsoft = dataset.load()

db_D_holistic = feature_extractor.compute_features(imgs_db)
q_D_holistic = feature_extractor.compute_features(imgs_q)

db_D_holistic = db_D_holistic / np.linalg.norm(db_D_holistic, axis=1, keepdims=True)
q_D_holistic = q_D_holistic / np.linalg.norm(q_D_holistic, axis=1, keepdims=True)
S = np.matmul(db_D_holistic, q_D_holistic.transpose())
```

How it works. The file loads images, converts them into descriptors, normalizes those descriptors, and computes the similarity matrix S . Everything later in the benchmark path depends on S .

4.2 datasets/load_dataset.py

What it does. This file owns dataset loading and ground-truth generation for:

- GardensPoint
- StLucia
- SFU
- the custom CampusDataset

Most useful idea. All loaders return the same interface: reference images, query images, strict ground truth, and soft ground truth.

Listing 2: Shared dataset contract

```
class Dataset(ABC):
    @abstractmethod
    def load(self) -> Tuple[List[np.ndarray], List[np.ndarray], np.ndarray, np.ndarray]:
        pass
```

How it works. This shared contract is what lets demo.py and test_campus_dataset.py swap datasets without changing the rest of the evaluation code.

Listing 3: Campus ground-truth logic

```
if '-npm' in night_name.lower() or night_name.startswith('npm'):
    continue
if night_name.startswith('PXL_'):
    continue
```

```

match = re.match(r'(image\d+)', night_name)
if match:
    base_name = match.group(1)
    if base_name in day_names:
        i = day_names.index(base_name)
        GThard[i, j] = True

```

How it works. The campus loader uses filenames to decide which night images truly match day images and which should be treated as no-match queries.

4.3 matching/matching.py

What it does. This file converts similarity scores into binary match decisions.

Listing 4: Two core matching strategies

```

def best_match_per_query(S: np.ndarray) -> np.ndarray:
    i = np.argmax(S, axis=0)
    j = np.int64(range(len(i)))
    M = np.zeros_like(S, dtype='bool')
    M[i, j] = True
    return M

def thresholding(S: np.ndarray, thresh: Union[str, float]) -> np.ndarray:
    if thresh == 'auto':
        mu = np.median(S)
        sig = np.median(np.abs(S - mu)) / 0.675
        thresh = norm.ppf(1 - 1e-6, loc=mu, scale=sig)
    return S >= thresh

```

How it works. One strategy always chooses exactly one match per query. The other uses a threshold so a query can have zero, one, or many matches depending on confidence.

4.4 evaluation/metrics.py

What it does. This file computes the benchmark metrics.

Listing 5: How PR curves are built

```

for i in np.linspace(startV, endV, n_thresh):
    B = S >= i
    TP = np.count_nonzero(GT & B)
    FP = np.count_nonzero((~GT) & B)
    P.append(TP / (TP + FP))
    R.append(TP / GTP)

```

How it works. Instead of choosing one threshold once, the code sweeps across many thresholds and records precision and recall at each one. That produces the PR curve and the AUC.

4.5 test_campus_dataset.py

What it does. This file adapts the benchmark pipeline to the custom campus dataset.

Why it matters. It shows how to add a custom dataset without rewriting the whole evaluation stack.

Listing 6: Campus script follows the same flow as `demo.py`

```
dataset = CampusDataset(destination=args.dataset_dir)
imgs_db, imgs_q, GThard, GTsoft = dataset.load()

db_D_holistic = feature_extractor.compute_features(imgs_db)
q_D_holistic = feature_extractor.compute_features(imgs_q)
S = np.matmul(db_D_holistic, q_D_holistic.transpose())
```

How it works. The structure is intentionally similar to `demo.py`. The biggest difference is the campus loader and the saved campus-specific output files.

4.6 `live_vpr_test.py`

What it does. This is the top-level CLI for the live pipeline.

Why it matters. It does not implement most logic itself; it orchestrates the modules under `live_vpr/`.

Listing 7: The live CLI wires together modular components

```
from live_vpr import (
    LiveDisplay,
    LiveLocalizer,
    LiveReferenceRecorder,
    MapBuildConfig,
    MapBuilder,
    OpenCVFrameSource,
    load_reference_map,
)
```

How it works. This file is best understood as the glue layer. If you want to add a new mode or a new CLI flag, this is where you start.

4.7 `live_vpr/offline.py`

What it does. This file owns reference-map creation from images.

Listing 8: Core map-building loop

```
images = [_load_rgb_image(path, target_size) for path in image_paths]
descriptors = compute_global_descriptors(self.extractor, images)
descriptors = normalize_descriptors(descriptors)
reference_map = ReferenceMap(
    descriptors=descriptors,
    image_paths=[str(path.resolve()) for path in image_paths],
    metadata=metadata,
)
```

How it works. A map is just a normalized descriptor matrix plus metadata and image paths. This is the main transition from “folder of images” to “runtime-ready live map”.

4.8 `live_vpr/database.py`

What it does. This file defines the saved map format and serialization.

Listing 9: ReferenceMap is the live system’s central data object

```
@dataclass
class ReferenceMap:
    descriptors: np.ndarray
    image_paths: list[str]
    metadata: dict[str, Any]
```

How it works. The live runtime does not read raw folders directly. It loads this object from disk and then does similarity lookup against `descriptors`.

4.9 live_vpr/capture.py

What it does. This file owns traversal recording and frame sampling.

Listing 10: Recording first, building later

```
if recording:
    writer.write(frame)
    frame_count += 1

status_frame = self._render_overlay(frame, recording, frame_count)
```

Listing 11: Sampling a traversal video into reference frames

```
should_sample = current_time_s + (0.5 / source_fps) >= next_sample_time_s
if should_sample:
    cv2.imwrite(str(output_path), frame)
    saved_paths.append(str(output_path))
    next_sample_time_s += sample_interval_s
```

How it works. Recording and sampling are intentionally separate. That makes it easy to rebuild a map later with a different sampling rate.

4.10 live_vpr/online.py

What it does. This file owns the runtime localization step.

Listing 12: The live localization core

```
descriptor = compute_global_descriptors(self.extractor, [rgb_image])
descriptor = descriptor / (np.linalg.norm(descriptor, axis=1, keepdims=True) + 1e-8)
similarities = (self.reference_map.descriptors @ descriptor.T).reshape(-1)
top_indices = np.argsort(similarities)[-top_k:][::-1]
```

How it works. The live phase still performs VPR in the same conceptual way as the benchmark path. The difference is that it localizes one frame at a time instead of evaluating an entire query set.

4.11 live_vpr/sources.py

What it does. This file abstracts webcams, stream URLs, and source aliases.

Listing 13: Alias resolution happens before capture opens

```
def parse_capture_source(source: str | int, aliases_path: str | None = None) -> str | int
:
    aliases = load_source_aliases(aliases_path)
    while source in aliases and source not in visited:
        source = aliases[source].strip()
    return int(source) if source.isdigit() else source
```

How it works. This lets the rest of the pipeline treat 0, phone, and `http://...` as different ways of naming the same idea: a capture source.

4.12 live_vpr/ui.py

What it does. This file owns the live overlay and the saved per-inference report images.

Listing 14: The UI has separate live and export render paths

```
display = LiveDisplay(reference_map=reference_map, show_top_k=not args.hide_top_k)
rendered = display.render(...)
export_canvas = display.render_inference_report(...)
```

How it works. The live overlay is optimized for real-time display, while the saved report image is optimized for readability and documentation.

4.13 scripts/live_vpr_cli.sh

What it does. This is a thin shell wrapper around `live_vpr_test.py`.

Listing 15: The launcher maps short commands to Python modes

```
record-map)
exec "$PYTHON_BIN" "$ENTRYPOINT" \
  --mode build_live_map \
  --descriptor "$DEFAULT_DESCRIPTOR" \
  --map_path "$DEFAULT_MAP_PATH" \
  --source "$DEFAULT_SOURCE" \
  --recording_path "$DEFAULT_RECORDING_PATH" \
  --capture_dir "$DEFAULT_CAPTURE_DIR" \
  --sample_fps "$DEFAULT_SAMPLE_FPS" \
  "$@"
```

How it works. The shell script does not contain VPR logic. It just shortens the commands and provides convenient environment-variable defaults.

5 Extension Points

If you want to change something, start here:

Goal	File to edit first
Add a new dataset	<code>datasets/load_dataset.py</code>
Add a new descriptor	<code>feature_extraction/</code> and <code>live_vpr/extractors.py</code>
Change benchmark metrics	<code>evaluation/metrics.py</code>
Change match selection	<code>matching/matching.py</code>

Change live map format	<code>live_vpr/database.py</code>
Change traversal recording	<code>live_vpr/capture.py</code>
Change live localization logic	<code>live_vpr/online.py</code>
Change overlays / saved re-port images	<code>live_vpr/ui.py</code>
Change camera or stream handling	<code>live_vpr/sources.py</code>
Add a new live mode or CLI flag	<code>live_vpr_test.py</code>

6 Practical Mental Model

Almost everything in the repo reduces to the same conceptual loop:

image $\rightarrow$ descriptor $\rightarrow$ similarity $\rightarrow$ decision

The benchmark path adds evaluation metrics. The live path adds UI, capture sources, and saved reference maps. If you keep that mental model in mind, the rest of the codebase becomes much easier to navigate.